Tarea #3

Ponderación: 2 pts

Tiempo estimado: 5 horas

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
Laboratorio - Arquitectura de Computadores y Ensambladores 1 - A
Auxiliar Carlos Alfredo Barrientos López

1. Marco Formativo

1.1 Valores

Indique un valor y cómo se aplica en el laboratorio.

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	El estudiante asume la responsabilidad de escribir código correcto, bien documentado y funcional en ARM-64, entregando evidencia de ejecución y depuración que demuestre el esfuerzo y la calidad de su trabajo.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Desarrolla programas funcionales en lenguaje ensamblador ARM-64, aplicando el set de instrucciones completo, control de flujo y convenciones de llamada para construir soluciones modulares y eficientes que manipulen datos a nivel de registros y memoria.
Competencia Específica	1. Aplica el set de instrucciones ARM-64 utilizando instrucciones aritméticas, lógicas, de carga/almacenamiento, desplazamientos y rotaciones para construir programas funcionales que manipulen datos a nivel de registros y memoria. 2. Implementa mecanismos de control de flujo en ensamblador ARM-64 empleando banderas, selección condicional, ciclos y branching para desarrollar algoritmos con estructuras de decisión y repetición equivalentes a lenguajes de alto nivel. 3. Construye funciones reutilizables en ensamblador ARM-64 aplicando convenciones de llamada, manejo del stack y paso de parámetros para modularizar programas y facilitar su mantenimiento y escalabilidad.

1.3 Objetivo

Implementar dos algoritmos clásicos de ordenamiento (Bubble Sort y Selection Sort) completamente en lenguaje ensamblador ARM-64 puro, aplicando instrucciones aritméticas, lógicas y de carga/almacenamiento, estructuras de control de flujo con ciclos anidados y branching, y convenciones de llamada con manejo correcto del stack, para construir un

programa modular que ordene arreglos en memoria y demuestre dominio del set de instrucciones ARM-64 mediante evidencia de ejecución y depuración con GDB.

Verbos de acción aplicados: implementar, construir, aplicar, demostrar, depurar.

2. Material de apoyo

• ARM Architecture Reference Manual ARMv8 (A64 Instruction Set): https://developer.arm.com/documentation/ddi0487/latest

• GNU Assembler (GAS) documentation para ARM-64: https://sourceware.org/binutils/docs/as/

• GDB Debugging Guide: https://sourceware.org/gdb/current/onlinedocs/gdb/

• QEMU ARM64 Emulation Setup Guide: https://wiki.qemu.org/Documentation/Platforms/ARM

• Procedure Call Standard for the ARM 64-bit Architecture (AAPCS64): https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst

3. Actividad

El estudiante deberá implementar dos algoritmos de ordenamiento clásicos completamente en ensamblador ARM-64 puro, aplicando control de flujo, manejo de arreglos en memoria y uso correcto del stack.

Tecnologías y herramientas: Linux (Ubuntu ARM64 o QEMU ARM64), ensamblador GNU (GAS / as), GDB para depuración, Makefile para compilación.

Requerimientos:

1. Implementar Bubble Sort en ARM-64

- Recibir un arreglo de enteros de 64 bits almacenado en memoria.
- Ordenar el arreglo in-place usando ciclos anidados y branching.
- La función debe recibir la dirección base del arreglo y su tamaño por registros.

2. Implementar Selection Sort en ARM-64

- Mismas condiciones que Bubble Sort.
- Implementar como función separada con su propio stack frame.

3. Programa principal

- Definir un arreglo de al menos 10 elementos en la sección .data.
- Imprimir el arreglo antes y después de ordenar usando syscall write.
- Llamar ambas funciones de forma modular.

4. Requisitos técnicos

- Cada función debe preservar correctamente los registros usando el stack (caller/callee saved).
- El código debe estar comentado línea a línea explicando la función de cada instrucción.
- Entregar evidencia de ejecución y depuración con GDB (al menos un breakpoint dentro del ciclo de ordenamiento).

Entregables:

- Código fuente .s debidamente comentado.
- Makefile para compilación y enlazado del proyecto.
- Captura de pantalla del resultado de ejecución (arreglo antes y después de ordenar).
- Sesión GDB documentada con al menos un breakpoint activo dentro del ciclo de ordenamiento.

Formato de entrega: Repositorio en GitHub/GitLab o archivo comprimido (.zip) subido a la plataforma del curso, con estructura de carpetas organizadas (src/, screenshots/).

4. Cronograma

Acción	Número de Semana	Fecha
Asignación de tarea	Semana 6	13/10/2026
Entrega de Tarea	Semana 8	18/10/2026

5. Rúbrica de calificación

5.1. Requisitos para optar a Calificación

Para optar a calificación, el estudiante debe cumplir obligatoriamente con los siguientes requisitos:

1. Entregar el código fuente (.s) con comentarios línea a línea antes de la fecha límite.
2. El programa debe compilar sin errores usando el Makefile incluido.
3. Incluir captura de pantalla de ejecución correcta (arreglo antes y después de ordenar).
4. Presentar evidencia de sesión GDB con al menos un breakpoint dentro de un ciclo de ordenamiento.

5.2 Detalle de la Calificación

CRITERIO	DESCRIPCIÓN	PUNTEO
Implementación de Bubble Sort	Función Bubble Sort correctamente implementada en ARM-64 puro: recibe dirección base y tamaño por registros, utiliza ciclos anidados con branching, ordena el arreglo in-place con enteros de 64 bits. El resultado del ordenamiento es correcto.	25
Implementación de Selection Sort	Función Selection Sort implementada como función separada con su propio stack frame. Preserva correctamente los registros callee-saved, recibe parámetros por registros y el resultado de ordenamiento es correcto.	25
Programa principal y modularidad	El programa principal define un arreglo de al menos 10 elementos en .data, imprime el arreglo antes y después de ordenar usando syscall write, y llama ambas funciones de forma modular. Incluye Makefile funcional.	20
Documentación y comentarios	El código presenta comentarios línea a línea que explican claramente la función de cada instrucción ARM-64. Los comentarios son precisos, descriptivos y facilitan la comprensión del algoritmo.	15
Evidencia de depuración con GDB	Se presenta captura de sesión GDB con al menos un breakpoint activo dentro del ciclo de ordenamiento, mostrando el estado de los registros y la memoria en ese punto de ejecución.	15

TOTAL	Puntaje Total	**100**